

PSID Crisis Module Documentation

A non-technical guide to the notebook, code architecture, formulas, weights, and final selection logic

This report explains how the PSID crisis-module workflow turns a large bank of candidate questions into a short, deployable module. It walks through the notebook structure, the main code functions, the Ri and Pi formulas, the construct-priority weights, and the logic used to decide which questions stay in the Generic Core and which remain crisis-specific toggles.

52

total ranked questions

28

selected questions

29.17

selected minutes

2.452

average Pi

What the project does

- Loads the final ranked CSV and reuses it as the single source of truth for the notebook, dashboard, report, and questionnaire demos.
- Scores each question using a baseline utility-to-burden ratio, then improves that ranking with an enhanced priority score called Pi.
- Writes synchronized outputs: ranked CSV, summary JSON, dashboard payload, recommendation bundle, and figure files.
- Keeps the final selected module below the 30-minute ceiling while preserving high-value crisis content.

Validated headline results

- The final benchmark selects 28 questions out of 52 ranked candidates.
- The selected module takes 29.17 minutes, compared with 68.95 minutes for the full corpus.
- Average Ri is 2.079 and average Pi is 2.452.
- The pipeline currently produces 6 generic-core recommendations and 8 crisis-specific recommendations.

How The Notebook Documents The Workflow

The notebook is not a separate experimental branch. It mirrors the production-ranked workflow and refresh path.

Notebook sections in plain language

1. Objective

Notebook section used to explain the current production workflow.

2. 2. Final Corpus Snapshot

Notebook section used to explain the current production workflow.

3. 3. Top Ranked Question Review

Notebook section used to explain the current production workflow.

4. 4. Selected Module Composition

Notebook section used to explain the current production workflow.

5. 5. Construct Coverage in the Selected Module

Notebook section used to explain the current production workflow.

6. 6. Stakeholder Deliverables

Notebook section used to explain the current production workflow.

7. 7. Figure Outputs and Supporting Bundles

Notebook section used to explain the current production workflow.

8. 8. Selected Question Inspection

Notebook section used to explain the current production workflow.

9. 9. Validation Checks

Notebook section used to explain the current production workflow.

10. 10. Refresh the Production Artifacts

Notebook section used to explain the current production workflow.

11. 11. Refresh Checklist

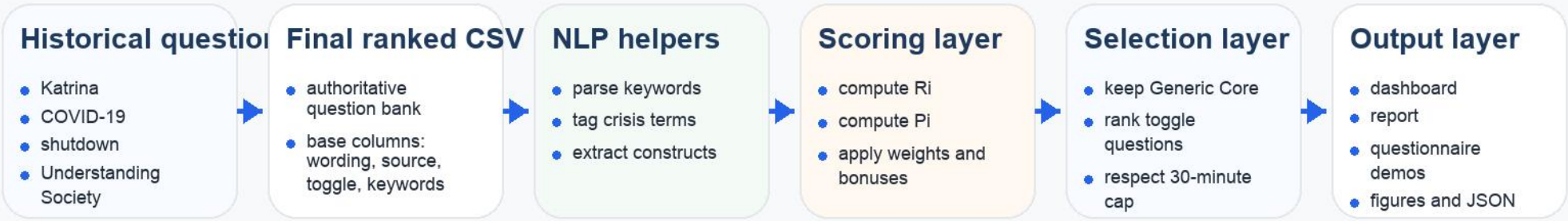
Notebook section used to explain the current production workflow.

12. 12. Final Summary

The important point for a non-technical reader is that the notebook is a guided explanation and validation layer. It loads the same final CSV and calls the same generator used to refresh the dashboard, report, figures, and recommendation outputs.

Workflow Design Architecture

How raw question text becomes ranked outputs, recommendations, and stakeholder-facing artifacts



What each major function does

parse_keywords() Turns the stored keyword string into a clean list the code can use.	tag_keywords() Matches each keyword to the project taxonomy, such as Employment, Housing / Shelter, or Trauma / Health.
extract_constructs() Collects the construct labels for each question so the model knows which crisis dimensions that question covers.	load_dataset() Loads the CSV, derives selected flags and minutes, and sends the data into the enhanced scoring pipeline.
compute_augmented_scores() Calculates rarity, redundancy, construct bonus, portability, augmented utility, and final Pi.	build_all() Runs the full refresh path and writes the final stakeholder artifacts.

How toggle decisions work

- The model does not invent crisis types from scratch. Each row already belongs to a routing group such as Generic Core, Financial Crisis, or Pandemic / Disaster.
- The scoring system ranks all rows, then the final selection keeps the small always-on Generic Core and fills the remaining time with the highest-value crisis-specific rows.
- In the current final module, the Pandemic / Disaster toggle dominates because Katrina and COVID questions contribute many strong housing, trauma, and aid signals.

Formula Calculations In Plain Language

The scoring system first asks which questions are useful for the least burden, then improves that ranking for deployability

1. Baseline ranking: Ri

$$R_i = U_i / B_i$$

$$B_i = \max(0.10 * \text{word_count} + 0.20 * \text{complexity}, 0.1)$$

- Utility means how much crisis-relevant information a question gives us.
- Burden means how much time or effort that question costs a respondent.
- A short, high-value question gets a high Ri.
- A long or complicated question gets penalized through Bi.

2. Enhanced priority: Pi

$$P_i = \text{augmented_utility} / [B_i * (1 + 0.65 * \text{redundancy_scaled})]$$

$$\text{augmented_utility} = U_i * (1 + 0.32 * \text{idf_scaled} + 0.24 * \text{construct_scaled} + 0.12 * \text{richness_scaled} + \text{portability_bonus})$$

- 0.32 rewards rare wording, because unique language often signals unique information.
- 0.24 rewards stronger construct bonus, so questions covering more important crisis dimensions rise.
- 0.12 rewards richness, meaning one question covering several important ideas gets extra credit.
- 0.65 penalizes redundancy, so near-duplicate questions do not crowd out the shortlist.

A non-technical way to read the formulas

- Ri asks: how much useful information do we get per unit of respondent effort?
- Pi asks: after we account for rarity, construct strength, portability, and repetition, which questions are the best ones to keep?
- This means a question can have a solid Ri but still move down if it is repetitive, too source-specific, or weak on important constructs.
- Likewise, a question can move up if it covers a high-priority construct in a clean and portable way.

Construct Bonus, Weights, And Why They Matter

The construct bonus is where rarity, priority, and richness come together

How `construct_bonus()` is computed

- **Rarity:** the model checks how often each construct appears across all questions. Less common constructs receive more credit.
- **Priority:** the model looks up the construct weights in the `CONSTRUCT_PRIORITY` dictionary and averages them for the constructs attached to the question.
- **Richness:** the model counts how many unique constructs a question covers and adds a small bonus for broader coverage.

```
rarity = mean(1 / construct_frequency) * N
priority = mean(CONSTRUCT_PRIORITY[name])
richness = len(unique_constructs)
construct_bonus = rarity + priority + 0.08 * richness
```

In plain English: a question gets extra credit if it touches a rare construct, touches a high-priority construct, or covers several important constructs at once.

Construct-priority weights



Portability bonus and redundancy penalty

- **Portability bonus** rewards wording that can be reused across crisis types. Generic-core questions with non-source-specific wording get the biggest bonus.
- **Redundancy penalty** lowers the score of questions that are too similar to other questions, using cosine similarity over TF-IDF vectors.
- This matters because the project is not trying to keep every good question. It is trying to keep a compact set of distinct, high-value questions.

Generic Core and no source-specific term -> 0.16
Generic Core and source-specific term -> 0.03
Toggle item and no source-specific term -> 0.08
Toggle item and source-specific term -> 0.02

Examples A Non-Technical Audience Can Follow

These examples show how the same scoring logic treats a generic-core question, a financial toggle question, and a pandemic/disaster question

Generic Core example	Financial Crisis toggle example	Pandemic / Disaster example
Have you experienced any financial difficulties because of the crisis? - Source: COVID-19 - Toggle category: Generic Core - Constructs: Financial Coping - Ri = 5.667 - Pi = 7.187	How did your household manage financial difficulties caused by the shutdown or crisis? - Source: Govt Shutdown Crisis - Toggle category: Toggle: Financial Crisis - Constructs: Financial Coping, Government Aid - Ri = 3.719 - Pi = 4.592	Did you lose earnings because of the pandemic? - Source: COVID-19 - Toggle category: Toggle: Pandemic / Disaster - Constructs: Economic / Income, Trauma / Health - Ri = 5.667 - Pi = 7.809
This question stays in the always-on core because it is short, easy to ask, and useful across almost any crisis type.	This question is longer, but it stays valuable because it captures how households actually cope with financial shock, not just whether hardship happened.	This question scores very highly because it is compact, clear, and covers strong crisis exposure signals without much burden.
Why this is helpful for a non-technical reader: - You can see that high-ranking questions are not always the longest or most detailed ones. - The model prefers questions that say something important clearly and without repeating what other questions already capture.	Why this is helpful for a non-technical reader: - You can see that high-ranking questions are not always the longest or most detailed ones. - The model prefers questions that say something important clearly and without repeating what other questions already capture.	Why this is helpful for a non-technical reader: - You can see that high-ranking questions are not always the longest or most detailed ones. - The model prefers questions that say something important clearly and without repeating what other questions already capture.

How The Final Module Is Selected And Refreshed

The workflow keeps a compact always-on core, then spends the remaining time budget on the strongest crisis-specific questions

Selection logic in plain language

- The Generic Core is intentionally small because it must work across many crisis types.
- The model then fills the remaining time budget with the highest-value crisis-specific questions.
- In the current final module, the Pandemic / Disaster bank wins most of that space because Katrina and COVID contribute many strong housing, trauma, and aid items.
- The Financial Crisis toggle is represented by the strongest available coping question in the current scored bank.

- Final result: 28 selected questions in 29.17 minutes.
- Generic Core selected: 7 questions.
- Financial Crisis selected: 1 question.
- Pandemic / Disaster selected: 20 questions.

What build_all() refreshes

- writes the ranked CSV with enhanced scoring fields
- writes psid_artifact_summary.json with headline benchmark values
- writes psid_dashboard_data.js for the browser dashboard
- regenerates the five main figures
- keeps the dashboard, report, notebook, and demos aligned to the same ranked-question workflow

```
build_all() -> _style_matplotlib() -> load_dataset() -> build_summary(df) -> write_ranked_csv(df) -> write_dashboard_data(df, summary) -> plot_* functions -> save_aliases()
```

This is why the notebook emphasizes build_all() as the only supported refresh path: it reduces drift and ensures that all public-facing artifacts point back to the same final ranked CSV.